

Context Statement

This memo aims to retrospectively examine natural disaster relief management with a particular focus on equity. Specifically, by analyzing the distribution of grants awarded following Hurricane Katrina in 2005, our aim is to help reform programs to better serve those in need. This memo aims to inform disaster aid administrators, who ultimately make decisions on the recipients and amounts associated with grant funding. In particular, with the memo, we address the director of the Louisiana Office for Community Developments, whose department is responsible for leading rebuilding efforts following natural disasters in the state of Louisiana, a hurricane-prone state. Our research is focused around the following two questions:

- What proportion of total property damage is recouped in disaster relief funding, and how does this vary by income status?
- What is the average amount of excess benefits awarded to citizens, and how does this total vary between insured and uninsured individuals?

By answering the questions above, we aim to spotlight and correct the inequity in relief aid, where lower-income populations are receiving inequitable treatment as they attempt to rebuild. This helps serve as a case study to rethink and retool disaster relief distribution such that vulnerable populations are prioritized in future recovery efforts.

Executive Summary / TL;DR

Following hurricanes Katrina and Rita in 2005, Louisiana launched the Road Home Program to help homeowners rebuild or repair storm-damaged property. However, with controversies in equitable allocation, this case study offers valuable insights into future program design. Learning from the mistakes of previous allocation mishaps, our study uncovers income disparities in aid allocation which should be resolved in future programs in order to provide fair, unbiased, and equitable outcomes for all residents.

By analyzing the state's own grant data, we demonstrate that the distribution of disaster relief funding was regressive, aiding homeowners with alternative means of aid, such as insurance payouts, rather than prioritizing homeowners without alternative means of aid. This leads to excess funding awarded to those who do not need it, and takes away timely funding from those who may need it most. We also demonstrate that the program's reliance on pre-storm property values, rather than actual damage assessments, systematically disadvantaged low-income homeowners whose properties were worth less before the storms.

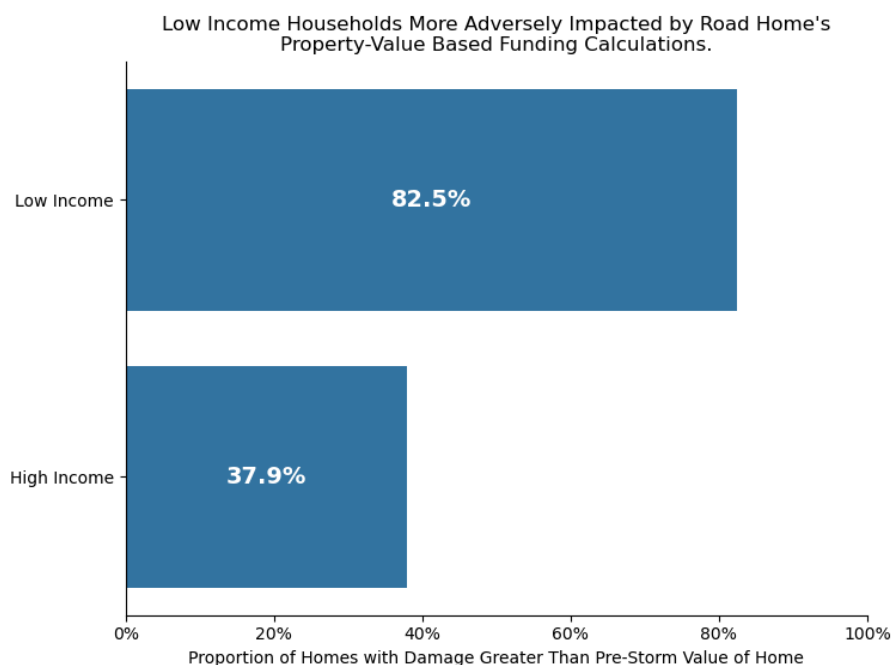
To prevent perpetuating inequalities during emergency situations, we advise disaster relief administrators to use damage estimates rather than property values when calculating aid distributions. Furthermore, we would advise administrators to better coordinate with insurance funding to eliminate wasteful, excess benefits and the redirection of such funding to address disproportionate gaps in coverage of damages for lower-income residents.

Louisiana Underfunded Poorer Populations in Recovery Efforts

In Louisiana, one of the most hurricane-prone states in the U.S, the equitable distribution of relief funds is not just a matter of fairness, but one of long-term community resilience. When aid is distributed inequitably, recovery slows for vulnerable populations and additional burden is placed on state and local agencies to fill unmet needs. Therefore, we seek to understand if Louisiana's Road Home Program equitably provided compensation to offset the amount of damage incurred by residents.

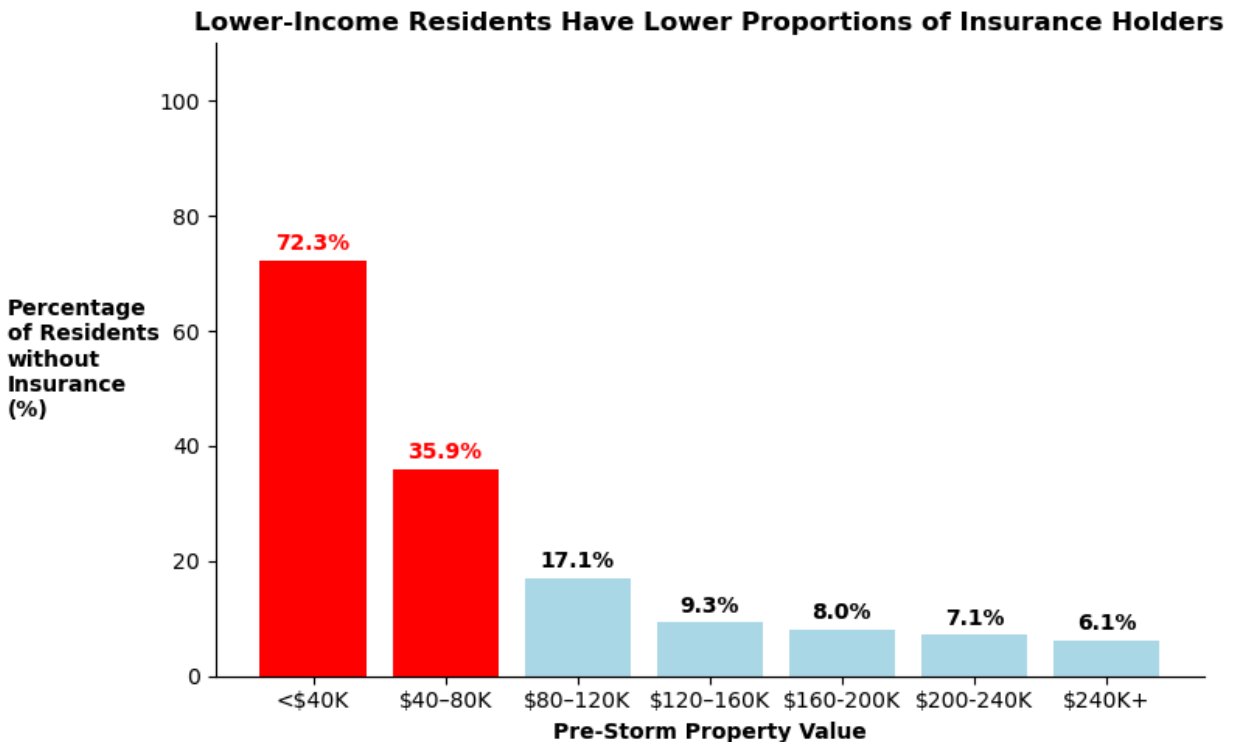
By examining how the Louisiana Road Home Program, a program set up to help homeowners rebuild or repair storm-damaged property, allocated grants after Hurricanes Katrina and Rita, our research found that lower property value homeowners had a lower proportion of their hurricane damage covered by the Road Home grants when compared to homeowners of property with higher property value. The program's grant formula was based on the lower of a property's pre-storm value or its damage assessment, meaning that poorer residents with low property value received less money than necessary to cover damages, even when their homes were completely destroyed.

As the chart below illustrates, 82.5% of low-income households suffered damages that exceeded the value of their homes, compared to 37.9% of higher income households, leaving low-income households with significantly less funding to rebuild. Since funding calculations used pre-storm home values, higher-value homeowners often had damages that were less than their comparatively higher pre-storm property values, allowing their grants to fully cover repair costs while leaving poorer families unable to fully cover damages. Future disaster relief administrations should pivot away from this grant model based on pre-storm value and reference damages instead in order to equitably help residents rebuild and recover.



Residents with Insurance Policies Receive Excess Benefits

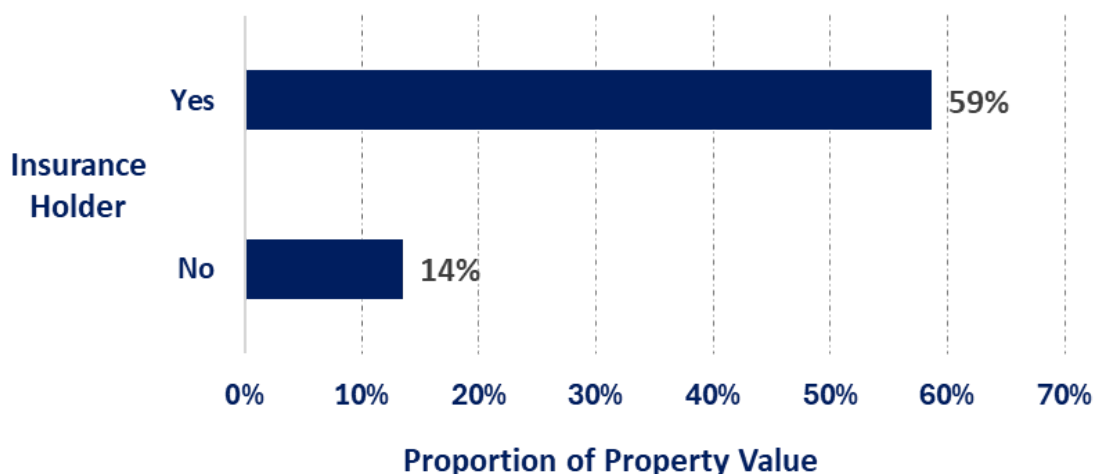
The insufficiency of funding to cover damages for lower-income residents only captures half the inequality story. To obtain the full picture and rectify the inequality gap in program payouts, it is necessary to analyze payouts awarded to wealthier residents. Our analysis pivots to the other half, and finds that wealthier residents received excess funding from the Road Home grant program due to duplicative benefits from the Road Home program and private insurance. The below graph illustrates that fewer residents with higher property values lacked flood and storm insurance when compared to their counterparts with lower property values.



Given that the proportion of non-insured individuals is significantly higher for populations with lower property value compared to those with higher property value, such policies stood to generate an outsized gap in disaster relief. With fewer lower-income residents enrolled in storm and flood insurance, our study also found, on average, those with insurance received almost six times the duplicative benefits, or excessive grant money, compared to residents without insurance.

The below chart shows that government programs did not account for insurance payouts, leading to insurance holders enjoying over 50% excess benefits compared to their pre-storm property values. For future policy design, this illustrates that government programs should coordinate with insurance agencies on total payout to ensure sufficient need exists for funds. Additionally, when combined with the prior finding that 82.5% of lower-income residents were uncompensated for total damage based on a grant formula tied to property value, such funds should be prioritized to assist residents with unmet damages rather than excessively compensating those whose insurance policies cover such damage.

Insurance Holders Receive Over Half of Their Property Value in Excess Benefits on Average



Takeaways

Our findings highlight that the Road Home Program’s reliance on pre-storm property values systematically disadvantaged low-income homeowners. Because grant amounts were capped by the lower of property value or damage, homeowners with less valuable properties received less assistance—even when their homes experienced higher amounts of damage.

Therefore, we recommend that future disaster relief programs base grant calculations on verified damage assessments rather than pre-storm property values. This approach ensures that aid aligns with actual losses, not with preexisting economic disparities.

Furthermore, our analysis shows that many higher-income households received excess funding due to overlapping sources of assistance, such as insurance payouts and federal aid. This inefficiency not only misdirects limited funds but also contributes to delays for households still waiting for sufficient support.

To prevent duplicative payments and redirect excess resources toward those with unmet needs, we recommend improved coordination between state disaster agencies and private insurers. Such coordination would allow relief programs to identify overlapping aid in real time and reallocate funds where they are most needed to cover damage.

Taken together, these adjustments would help future rebuilding efforts prioritize need over asset value, leading to more equitable recovery outcomes and stronger, more resilient communities across Louisiana’s Gulf Coast.

```
In [2]: import plotly.figure_factory as ff
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from matplotlib.ticker import FuncFormatter
```

Loading Datasets for GeoPanads, loan, and poverty information

```
In [3]: # Load the loan dataset
df_loan = pd.read_csv(
    "https://raw.githubusercontent.com/wujenny214/Policy-Study-into-Grant-Assistance-for-Post-Hurricane-Rebuilding-
```

```
)

df_loan = df_loan[df_loan["GIS State"] == "LA"]
df_loan = df_loan[df_loan["ARS File (Yes/No)"] == "N"]
df_loan = df_loan[df_loan["Current Damage Assessment"] != 9999999.0]
```

/var/folders/xm/yhmrwng57qq9wlcywlm89pxh0000gn/T/ipykernel_98792/4198342093.py:2: DtypeWarning: Columns (13,15) have mixed types. Specify dtype option on import or set low_memory=False.

```
df_loan = pd.read_csv(
```

```
In [4]: # Load SAIPE 2004 Louisiana county-level poverty data
```

```
colspeccs = [
    (0, 2), # 1-2 FIPS State
    (3, 6), # 4-6 FIPS County
    # (7, 15), # 8-15 Est all ages in poverty
    # (16, 24), # 17-24 LCI
    # (25, 33), # 26-33 UCI
    (34, 38), # 35-38 % all ages est
    # (39, 43), # 40-43 % all ages LCI
    # (44, 48), # 45-48 % all ages UCI
    # (49, 57), # 50-57 Est age 0-17
    # (58, 66), # 59-66 LCI
    # (67, 75), # 68-75 UCI
    # (76, 80), # 77-80 % 0-17 est
    # (81, 85), # 82-85 % 0-17 LCI
    # (86, 90), # 87-90 % 0-17 UCI
    # (91, 99), # 92-99 Est related children 5-17 in families
    # (100, 108), # 101-108 LCI
    # (109, 117), # 110-117 UCI
    # (118, 122), # 119-122 % related 5-17 est
    # (123, 127), # 124-127 % related 5-17 LCI
    # (128, 132), # 129-132 % related 5-17 UCI
    # (133, 139), # 134-139 Median HH income
    # (140, 146), # 141-146 Median HH income LCI
    # (147, 153), # 148-153 Median HH income UCI
    # (154, 161), # Estimate of people under age 5 in poverty
    # (162, 169), # 90% confidence interval lower bound of estimate of people under age 5 in poverty
    # (170, 177), # 90% confidence interval upper bound of estimate of people under age 5 in poverty
    # (178, 182), # Estimated percent of people under age 5 in poverty
    # (183, 187), # 90% confidence interval lower bound of estimate of percent of people under age 5 in poverty
    # (189, 192), # 90% confidence interval upper bound of estimate of percent of people under age 5 in poverty
    (193, 238), # State or county name
    # (239, 241), # Two-letter Postal State abbreviation
    # (242, 266), # A tag indicating the file name and date of creation
]
```

```
names = [
    "state_fips",
    "county_fips",
    # "all_ages_pov_est",
    # "all_ages_pov_l",
    # "all_ages_pov_u",
    "all_ages_pov_pct_est",
    # "all_ages_pov_pct_l",
    # "all_ages_pov_pct_u",
    # "age0_17_pov_est",
    # "age0_17_pov_l",
    # "age0_17_pov_u",
    # "age0_17_pov_pct_est",
    # "age0_17_pov_pct_l",
    # "age0_17_pov_pct_u",
    # "rel5_17_pov_est",
    # "rel5_17_pov_l",
    # "rel5_17_pov_u",
]
```

```

# "rel5_17_pov_u",
# "rel5_17_pov_pct_est",
# "rel5_17_pov_pct_l",
# "rel5_17_pov_pct_u",
# "med_hh_income_est",
# "med_hh_income_l",
# "med_hh_income_u",
# "est_ppl_under5pov",
# "lci_ppl_under5pov",
# "uci_ppl_under5pov",
# "pct_ppl_under5pov_est",
# "pct_ppl_under5pov_l",
# "pct_ppl_under5pov_u",
"parish",
# "state",
# "file_tag",
]

# Read the fixed-width file
url = "https://www2.census.gov/programs-surveys/saipe/datasets/2004/2004-state-and-county/est04-la.txt"
df_poverty = pd.read_fwf(url, colspecs=colspecs, names=names, dtype=str)

```

df_poverty

```

Out [4]:

```

	state_fips	county_fips	all_ages_pov_pct_est	parish
0	22	0	19.2	Louisiana
1	22	1	21.0	Acadia Parish
2	22	3	22.0	Allen Parish
3	22	5	12.0	Ascension Parish
4	22	7	19.3	Assumption Parish
...	...	...	...	...
60	22	119	19.0	Webster Parish
61	22	121	16.8	West Baton Rouge Parish
62	22	123	22.7	West Carroll Parish
63	22	125	20.4	West Feliciana Parish
64	22	127	22.8	Winn Parish

65 rows x 4 columns

```

In [5]:
# # Construct the FIPS codes directly and save them into the dataframe
# df_poverty["FIPS"] = df_poverty.apply(
#     lambda row: row["state_fips"]
#     + ("00" if len(row["county_fips"]) == 1 else "0")
#     + row["county_fips"],
#     axis=1,
# )

# (ALT method) Initialize an empty list to store the FIPS codes
fips_codes = []

# Iterate through each row in the dataframe
for i, row in df_poverty.iterrows():
    # Check the length of the county_fips and construct the FIPS code accordingly
    if len(row["county_fips"]) == 1:
        fips_code = row["state_fips"] + "00" + row["county_fips"]
    elif len(row["county_fips"]) == 2:
        fips_code = row["state_fips"] + "0" + row["county_fips"]
    else:
        fips_code = row["state_fips"] + row["county_fips"]
    fips_codes.append(fips_code)

# Add the constructed FIPS codes as a new column in the dataframe
df_poverty["FIPS"] = fips_codes

df_poverty = df_poverty[["FIPS", "parish", "all_ages_pov_pct_est"]].copy()
df_poverty = df_poverty[1:]
df_poverty

```

```
Out [5]:
```

	FIPS	parish	all_ages_pov_pct_est
1	22001	Acadia Parish	21.0
2	22003	Allen Parish	22.0
3	22005	Ascension Parish	12.0
4	22007	Assumption Parish	19.3
5	22009	Avoyelles Parish	24.8
...	...	...	...
60	22119	Webster Parish	19.0
61	22121	West Baton Rouge Parish	16.8
62	22123	West Carroll Parish	22.7
63	22125	West Feliciana Parish	20.4
64	22127	Winn Parish	22.8

64 rows x 3 columns

Plotting Louisiana

```
In [6]: # Download all US counties (2023, 1:5m cartographic boundaries) from Census
url = "https://www2.census.gov/geo/tiger/GENZ2023/shp/cb_2023_us_county_5m.zip"
counties = gpd.read_file(url)

# Keep only Louisiana (STATEFP = '22')
la = counties[counties["STATEFP"] == "22"].copy()

# Keep useful fields; GEOID is the full 5-digit county/parish FIPS
la = la[["GEOID", "STATEFP", "COUNTYFP", "NAME", "geometry"]]

# Save as GeoJSON (Leaflet/Mapbox/Folium/Plotly-ready)
la.to_file("louisiana_counties_fips.geojson", driver="GeoJSON")

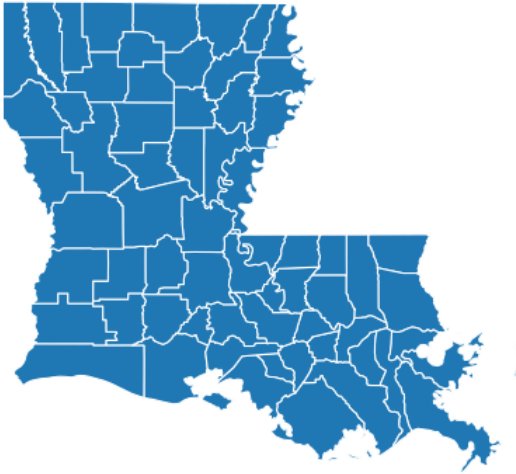
# Mapping Louisiana parishes
ax = la.plot(edgecolor="white", linewidth=0.8)
ax.set_axis_off()

la
```

```
Out [6]:
```

	GEOID	STATEFP	COUNTYFP	NAME	geometry
103	22061	22	061	Lincoln	POLYGON ((-92.88093 32.58526, -92.86661 32.607...
392	22095	22	095	St. John the Baptist	POLYGON ((-90.68727 30.0372, -90.66997 30.0476...
409	22029	22	029	Concordia	POLYGON ((-91.8633 31.35445, -91.84502 31.3759...
570	22003	22	003	Allen	POLYGON ((-93.13119 30.51828, -93.13081 30.564...
628	22093	22	093	St. James	POLYGON ((-90.96374 30.06683, -90.93559 30.085...
...	...	...	...	...	...
3095	22083	22	083	Richland	POLYGON ((-92.03117 32.2814, -92.01982 32.2976...
3118	22037	22	037	East Feliciana	POLYGON ((-91.32071 30.69942, -91.2931 30.7155...
3126	22051	22	051	Jefferson	POLYGON ((-90.28006 29.99171, -90.27986 29.998...
3153	22049	22	049	Jackson	POLYGON ((-92.81715 32.1617, -92.79882 32.1813...
3160	22115	22	115	Vernon	POLYGON ((-93.56976 30.99672, -93.56798 31.001...

64 rows x 5 columns



```
In [7]: # Concatenate the la dataframe with the df_poverty dataframe to include the geometry variable
df_poverty_with_geometry = pd.concat(
    [df_poverty.set_index("FIPS"), la.set_index("GEOID")["geometry"]],
    axis=1,
    join="inner",
).reset_index()

df_poverty_with_geometry
```

```
Out [7]:
```

	index	parish	all_ages_pov_pct_est	geometry
0	22001	Acadia Parish	21.0	POLYGON ((-92.63216 30.31429, -92.6195 30.3400...
1	22003	Allen Parish	22.0	POLYGON ((-93.13119 30.51828, -93.13081 30.564...
2	22005	Ascension Parish	12.0	POLYGON ((-91.10656 30.10852, -91.08983 30.108...
3	22007	Assumption Parish	19.3	POLYGON ((-91.25894 30.00067, -91.23635 30.000...
4	22009	Avoyelles Parish	24.8	POLYGON ((-92.28074 30.96507, -92.2086 30.9621...
...	...	...	...	...
59	22119	Webster Parish	19.0	POLYGON ((-93.51632 33.00254, -93.52099 33.018...
60	22121	West Baton Rouge Parish	16.8	POLYGON ((-91.48173 30.51103, -91.41482 30.511...
61	22123	West Carroll Parish	22.7	POLYGON ((-91.63888 32.70247, -91.61772 32.720...
62	22125	West Feliciana Parish	20.4	MULTIPOLYGON (((-91.66251 30.97036, -91.64877 ...
63	22127	Winn Parish	22.8	POLYGON ((-92.97604 31.71242, -92.9686 31.7310...

64 rows x 4 columns

```
In [8]: import pandas as pd
import matplotlib.pyplot as plt

# Create bins manually
bins = [0, 5, 10, 15, 20, 25, 100]
labels = ["0-5%", "6-10%", "11-15%", "16-20%", "21-25%", "25%+"]

# Ensure the column is numeric before applying pd.cut
df_poverty_with_geometry["all_ages_pov_pct_est"] = pd.to_numeric(
    df_poverty_with_geometry["all_ages_pov_pct_est"], errors="coerce"
)

df_poverty_with_geometry["pov_bin"] = pd.cut(
    df_poverty_with_geometry["all_ages_pov_pct_est"],
    bins=bins,
    labels=labels,
    include_lowest=True,
)

df_poverty_with_geometry = gpd.GeoDataFrame(
    df_poverty_with_geometry, geometry="geometry", crs="EPSG:4269"
)

df_poverty_with_geometry
```

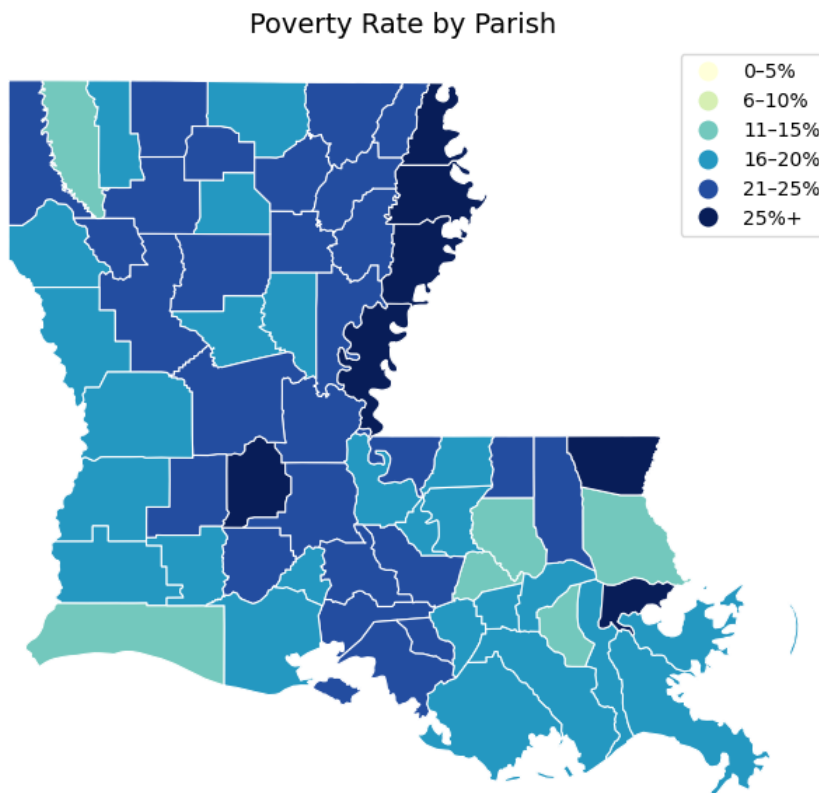


```
Out [8]:
```

	index	parish	all_ages_pov_pct_est	geometry	pov_bin
0	22001	Acadia Parish	21.0	POLYGON ((-92.63216 30.31429, -92.6195 30.3400...	21-25%
1	22003	Allen Parish	22.0	POLYGON ((-93.13119 30.51828, -93.13081 30.564...	21-25%
2	22005	Ascension Parish	12.0	POLYGON ((-91.10656 30.10852, -91.08983 30.108...	11-15%
3	22007	Assumption Parish	19.3	POLYGON ((-91.25894 30.00067, -91.23635 30.000...	16-20%
4	22009	Avoyelles Parish	24.8	POLYGON ((-92.28074 30.96507, -92.2086 30.9621...	21-25%
...	...	...	...	...	...
59	22119	Webster Parish	19.0	POLYGON ((-93.51632 33.00254, -93.52099 33.018...	16-20%
60	22121	West Baton Rouge Parish	16.8	POLYGON ((-91.48173 30.51103, -91.41482 30.511...	16-20%
61	22123	West Carroll Parish	22.7	POLYGON ((-91.63888 32.70247, -91.61772 32.720...	21-25%
62	22125	West Feliciana Parish	20.4	MULTIPOLYGON (((-91.66251 30.97036, -91.64877 ...	21-25%
63	22127	Winn Parish	22.8	POLYGON ((-92.97604 31.71242, -92.9686 31.7310...	21-25%

64 rows x 5 columns

```
In [9]: # Plot with grey for missing
fig, ax = plt.subplots(figsize=(8, 8))
df_poverty_with_geometry.plot(
    column="pov_bin",
    cmap="YlGnBu",
    linewidth=0.8,
    edgecolor="white",
    legend=True,
    ax=ax,
    missing_kwds={
        "color": "lightgrey",
        "edgecolor": "white",
        "hatch": "///",
        "label": "No data",
    },
)
ax.set_title("Poverty Rate by Parish", fontsize=14)
ax.axis("off")
plt.show()
```



```

In [10]: # To get accurate figures for the net grant amounts, the difference between "current" (Current Total DOB Amount) a
# be subtracted from the sum of the compensation and additional compensation grants (TOTAL_CLOSING_AMOUNT).

# Difference between "current" (Current Total DOB Amount) and "closing" (Closing Total DOB Amount) DOB
df_loan["delta"] = (
    df_loan["Current Total DOB Amount (no Legal Fees removed)"]
    - df_loan["Closing Total DOB Amount"]
)

# Calculate gross average loan amount
df_loan["gross_loan_amount"] = df_loan["TOTAL_CLOSING_AMOUNT"] - df_loan["delta"]

df_loan.head()

```

Out[10]:

	Unnamed: 0	Structure Type	GIS City	GIS State	GIS Zip	PARISH	Closing Option	TOTAL_CLOSING_AMOUNT	Total CG Amount	Total ACG Amunt	...	PSV at Closing
0	9613.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	150000.00	95213.27	39036.73	...	172000.0
1	21349.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	2.0	150000.00	150000.00	0.00	...	250000.0
2	25325.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	3450.14	3450.14	0.00	...	240000.0
3	27236.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	10665.59	10665.59	0.00	...	136000.0
4	36177.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	131947.98	101947.98	0.00	...	345000.0

5 rows x 39 columns

```

In [11]: from matplotlib.ticker import FuncFormatter

# Filter the dataframe for Low Income rows
low_income_df = df_loan[df_loan["Total ACG Amunt"] > 0]

# Filter the dataframe for High Income rows
high_income_df = df_loan[df_loan["Total ACG Amunt"] == 0]

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Define formatter for dollar amounts with commas
formatter = FuncFormatter(lambda x, _: f"${x:,.0f}")

# Plot Low Income dots
ax1.scatter(
    low_income_df["Current PSV"],
    low_income_df["Current Damage Assessment"],
    c="red",
    alpha=0.4,
    s=100
)

```

```

)
ax1.plot(
    [low_income_df["Current PSV"].min(), low_income_df["Current PSV"].max()],
    [low_income_df["Current PSV"].min(), low_income_df["Current PSV"].max()],
    color="black",
    linestyle="--",
)
ax1.set_xlabel("Pre-Storm Value ($)")
ax1.set_ylabel("Damage Assessment ($)")
ax1.set_title("Low Property Value Households", fontsize=12)
ax1.xaxis.set_major_formatter(formatter)
ax1.yaxis.set_major_formatter(formatter)

# Plot High Income
ax2.scatter(
    high_income_df["Current PSV"],
    high_income_df["Current Damage Assessment"],
    c="blue",
    alpha=0.4,
    s=10,
)
ax2.plot(
    [high_income_df["Current PSV"].min(), high_income_df["Current PSV"].max()],
    [high_income_df["Current PSV"].min(), high_income_df["Current PSV"].max()],
    color="black",
    linestyle="--",
)
ax2.set_xlabel("Pre-Storm Value ($)")
ax2.set_title("High Property Value Households", fontsize=12)
ax2.xaxis.set_major_formatter(formatter)
ax2.yaxis.set_major_formatter(formatter)

# Remove gridlines, legends, and extra text
ax1.grid(False)
ax2.grid(False)
ax1.legend().remove()
ax2.legend().remove()

fig.suptitle(
    "Damage Outpaced Property Value for Low-Income Residents Under the Road Home Program\nLeading to Less Aid Distr
    fontsize=14,
    y=1.1,
)
fig.text(
    0.5,
    0.95,
    "Because Road Home grants were based on pre-storm property values rather than actual damage, \n low-income home
    ha="center",
    fontsize=10,
)

# Simplify layout
plt.tight_layout()
plt.show()

```

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



```
In [12]: import matplotlib as mpl

mpl.rcParams.update(
    {
        "svg.fonttype": "none", # keep text editable in PPT
        "svg.image_inline": True, # embed raster parts
    }
)

import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# Filter the dataframe for Low Income rows
low_income_df = df_loan[df_loan["Total ACG Amunt"] > 0]
high_income_df = df_loan[df_loan["Total ACG Amunt"] == 0]

# plot
fig, ax = plt.subplots(figsize=(10, 8), dpi=300) # dpi controls rasterized quality
formatter = FuncFormatter(lambda x, _: f"${x:,.0f}")

# scatter layers rasterized so PPT can import
ax.scatter(
    high_income_df["Current PSV"],
    high_income_df["Current Damage Assessment"],
    c="red",
    alpha=0.4,
    s=10,
    label="High Property Value Households",
    rasterized=True,
)
ax.scatter(
    low_income_df["Current PSV"],
    low_income_df["Current Damage Assessment"],
    c="lightblue",
    edgecolors="white",
    alpha=0.3,
    s=10,
    label="Low Property Value Households",
    rasterized=True,
)

# diagonal reference line
all_psv = df_loan["Current PSV"]
ax.plot(
    [all_psv.min(), all_psv.max()],
    [all_psv.min(), all_psv.max()],
    color="black",
    linestyle="--",
    linewidth=1,
    label="Equal Value Line",
)

# limits, labels, formatting
max_damage = max(
    low_income_df["Current Damage Assessment"].max(),
    high_income_df["Current Damage Assessment"].max(),
)
```

```

)
max_psv = df_loan["Current PSV"].max()
ax.set_xlim(0, max_psv)
ax.set_ylim(0, max_damage)
ax.set_xlabel("Pre-Storm Value ($)", fontsize=11)
ax.set_ylabel("Damage Assessment ($)", fontsize=11)
ax.xaxis.set_major_formatter(formatter)
ax.yaxis.set_major_formatter(formatter)
ax.grid(False)

# titles
fig.suptitle(
    "Damage Outpaced Property Value for Low-Income Residents Under the Road Home Program\n"
    "Leading to Less Aid Distribution for Low-Income Households",
    fontsize=14,
    y=1.02,
)
fig.text(
    0.5,
    0.92,
    "Because Road Home grants were based on pre-storm property values rather than actual damage,\n"
    "low-income homeowners—whose homes were worth less before the storm—received less funding to rebuild.",
    ha="center",
    fontsize=10,
)

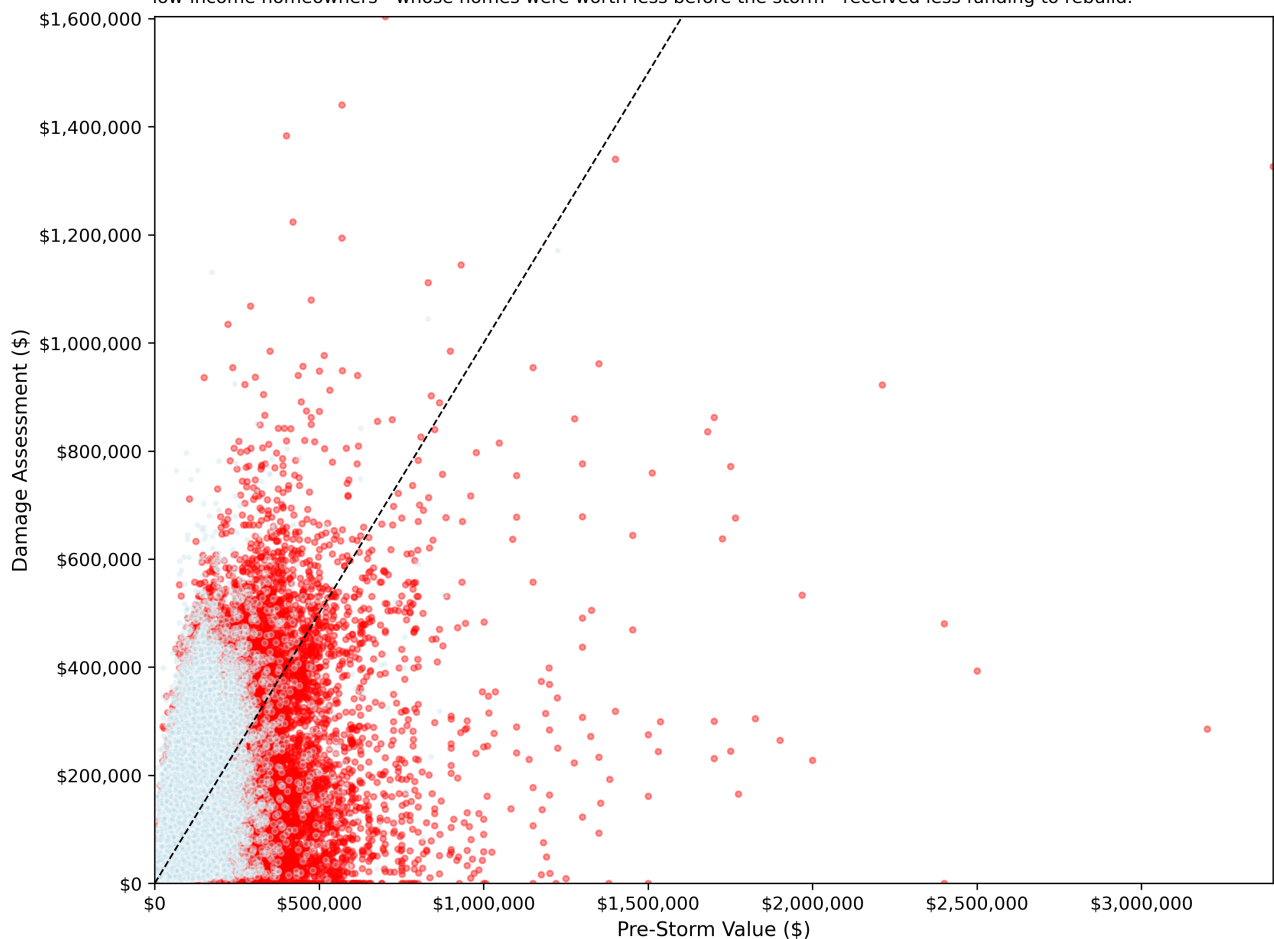
plt.tight_layout()

# save PPT-safe SVG (points are rasterized inside the SVG)
raw_svg = "jw_memo_plot01.svg"
fig.savefig(raw_svg, format="svg", transparent=True)

```

Damage Outpaced Property Value for Low-Income Residents Under the Road Home Program Leading to Less Aid Distribution for Low-Income Households

Because Road Home grants were based on pre-storm property values rather than actual damage,
low-income homeowners—whose homes were worth less before the storm—received less funding to rebuild.



```

In [13]: for i in df_loan.index:
          if df_loan.at[i, "Current Damage Assessment"] > df_loan.at[i, "Current PSV"]:
              df_loan.at[i, "page_psv"] = 1

```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
else:
    df_loan.at[i, "damage_psv"] = 0

df_loan.head()
```

Out[13]:

	Unnamed: 0	Structure Type	GIS City	GIS State	GIS Zip	PARISH	Closing Option	TOTAL_CLOSING_AMOUNT	Total CG Amount	Total ACG Amunt	...	Closer file - PSN Option With No ACG - excluding Sold Home Inelig Deceaser and file: closer with RHE & IMM only -23,91%
0	9613.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	150000.00	95213.27	39036.73	...	↑
1	21349.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	2.0	150000.00	150000.00	0.00	...	↑
2	25325.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	3450.14	3450.14	0.00	...	↑
3	27236.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	10665.59	10665.59	0.00	...	↑
4	36177.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	131947.98	101947.98	0.00	...	↑

5 rows x 40 columns

```
In [45]: # Filter the dataframe for Low Income rows
low_income_df = df_loan[df_loan["Total ACG Amunt"] > 0]
high_income_df = df_loan[df_loan["Total ACG Amunt"] == 0]

# Calculate proportions
low_income_proportion = low_income_df["damage_psv"].mean()
high_income_proportion = high_income_df["damage_psv"].mean()

# Create bar plot with flipped axes
proportions = [low_income_proportion, high_income_proportion]
labels = ["Low Income", "High Income"]

plt.figure(figsize=(8, 6))
bars = sns.barplot(y=labels, x=proportions, orient="h")

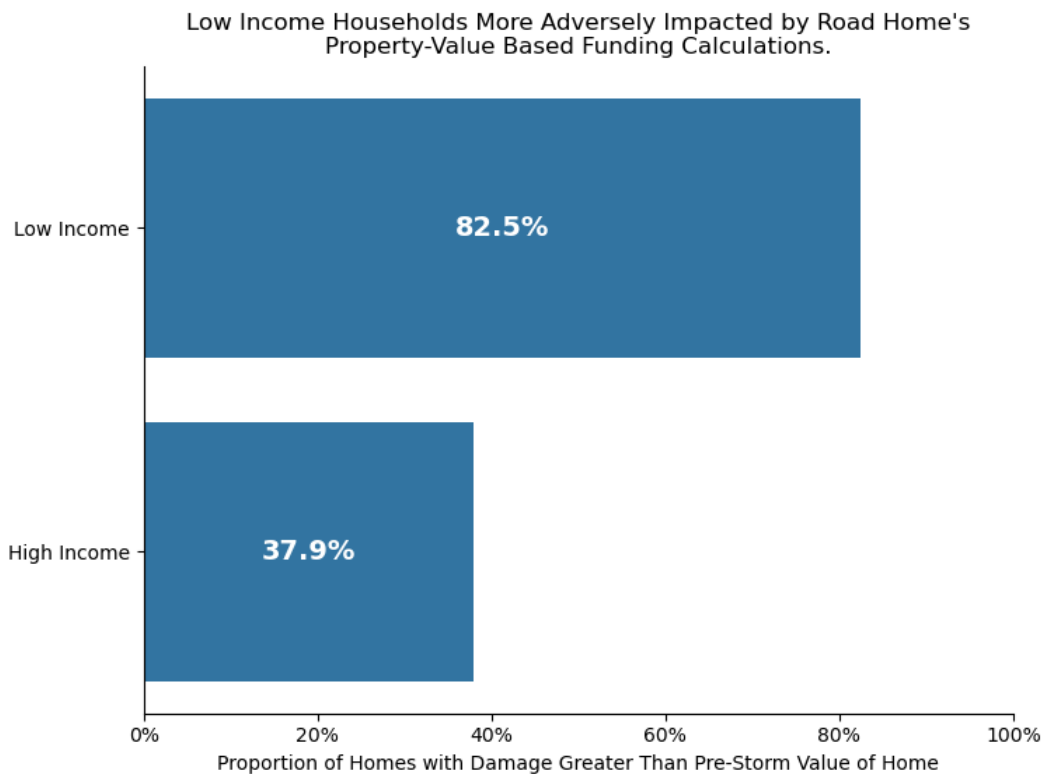
# Set x-axis limits to 0-100%
plt.xlim(0, 1.0)

# Format x-axis as percentage
plt.gca().xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: "{:.0%}".format(x)))

# Add percentage labels to the right of bars
for i, (bar, proportion) in enumerate(zip(bars.patches, proportions)):
    bars.text(
        proportion / 2,
        bar.get_y() + bar.get_height() / 2.0,
        f"{proportion:.1%}",
        ha="center",
        va="center",
        fontsize=14.
```

```
        fontweight="bold",
        color="white",
    )

    plt.xlabel("Proportion of Homes with Damage Greater Than Pre-Storm Value of Home")
    plt.ylabel("")
    plt.title(
        "Low Income Households More Adversely Impacted by Road Home's\nProperty-Value Based Funding Calculations."
    )
    plt.rcParams.update(
        {
            "svg.fonttype": "none", # keep text editable in PPT
        }
    )
    plt.savefig("jw_memo_plot02.svg", format="svg", transparent=True)
    bars.spines["top"].set_visible(False)
    bars.spines["right"].set_visible(False)
    plt.show()
```



Hurricane Katrina Review -Insurance

Import Variables

```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Bring in dataset

```
In [8]: roadhome = pd.read_csv(
    "https://raw.githubusercontent.com/wujenny214/Policy-Study-into-Grant-Assistance-for-Post-Hurricane-Relief")
```

/var/folders/xm/yhmrwng57qq9wlcylm89pxh0000gn/T/ipykernel_32129/320429160.py:1: DtypeWarning: Columns (13,15) have mixed types. Specify dtype option on import or set low_memory=False.

```
roadhome = pd.read_csv(
```

Clean data

```
In [9]: roadhome = roadhome[roadhome["GIS State"] == "LA"]
roadhome = roadhome[roadhome["ARS File (Yes/No)"] == "N"]
roadhome = roadhome[roadhome["Current Damage Assessment"] != 9999999.0]
```

```
In [10]: roadhome
```


Out [10]:

	Unnamed: 0	Structure Type	GIS City	GIS State	GIS Zip	PARISH	Closing Option	TOTAL_CLOSING_AMOUNT	Total CG Amount
0	9613.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	150000.00	95213.27
1	21349.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	2.0	150000.00	150000.00
2	25325.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	3450.14	3450.14
3	27236.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	10665.59	10665.59
4	36177.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	131947.98	101947.98
...	...	...	...	...	...	...	...	...	...
130048	44583.0	Mobile Home	OAKDALE	LA	71463-6000	Allen	1.0	26356.16	6997.46
130049	85246.0	Mobile Home	OAKDALE	LA	71463-8005	Evangeline	1.0	9015.20	9015.20
130050	121180.0	Single (including mobile home)	ZWOLLE	LA	71486-2603	Sabine	1.0	14802.42	14802.42
130051	124807.0	Single (including mobile home)	ZWOLLE	LA	71486-2761	Sabine	1.0	14471.91	14471.91
130052	97713.0	Mobile Home on Leased Land	ZWOLLE	LA	71486-3562	Sabine	1.0	9411.57	9411.57

121082 rows x 37 columns

Summarize the Pre-Storm value

```
In [11]: roadhome["Current PSV"].describe()
```

```
Out [11]: count    1.210820e+05
          mean     1.365587e+05
          std      9.773124e+04
          min      1.000000e+00
          25%      8.500000e+04
          50%      1.220000e+05
          75%      1.640000e+05
          max      3.400000e+06
          Name: Current PSV, dtype: float64
```

Checking the property value by quantile to identify lower-income residents, and associate with property

```
In [12]: roadhome["Current PSV"].quantile(0.85)
```

```
Out [12]: 203471.6849999999
```

Bin by property value to illustrate disparity in insurance

```
In [13]: bins = [0, 40000, 80000, 120000, 160000, 200000, 240000, float("inf")]
          labels = [
              "<$40K",
              "$40-80K",
              "$80-120K",
              "$120-160K",
              "$160-200K",
              "$200-240K",
              "$240K+",
          ]
```

```
In [14]: roadhome["property_bucket"] = pd.cut(
          roadhome["Current PSV"], bins=bins, labels=labels, include_lowest=True
          )
```

Segment by bucket

```
In [15]: roadhome["property_bucket"].value_counts()
```

```
Out [15]: property_bucket
$80-120K    31904
$120-160K   30461
$40-80K     14986
$160-200K   12822
<$40K       12356
$240K+      12063
$200-240K    6490
          Name: count, dtype: int64
```

Recode Insurance holders to represent "Y" with "1" and "N" with "0"

```
In [16]: roadhome["insurance_num"] = roadhome[
          "Applicant With\
          Current Insurance (Private and/or Flood) Y/N\
          "
          ].map({"Y": 1, "N": 0})
```

Prepare table to compute proportion of people with insurance by property value group

```
In [17]: avg_response = roadhome.groupby("property_bucket")["insurance_num"].mean().reset_index()

          print(avg_response)

          property_bucket  insurance_num
0          <$40K          0.277355
1          $40-80K          0.640598
2          $80-120K          0.829081
3          $120-160K          0.906930
4          $160-200K          0.919825
5          $200-240K          0.928968
6          $240K+           0.938821
```

```
/var/folders/xm/yhmrwng57qq9wlcylm89pxh0000gn/T/ipykernel_32129/2331860686.py:1: FutureWarning: The default
t of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future default and silence this warning.
    avg_response = roadhome.groupby("property_bucket")["insurance_num"].mean().reset_index()
```

```
In [18]: count_response = (
    roadhome.groupby("property_bucket")["insurance_num"].count().reset_index()
)

print(count_response)
```

	property_bucket	insurance_num
0	<\$40K	12356
1	\$40–80K	14986
2	\$80–120K	31904
3	\$120–160K	30461
4	\$160–200K	12822
5	\$200–240K	6490
6	\$240K+	12063

```
/var/folders/xm/yhmrwng57qq9wlcylm89pxh0000gn/T/ipykernel_32129/1853335242.py:2: FutureWarning: The default
t of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future default and silence this warning.
    roadhome.groupby("property_bucket")["insurance_num"].count().reset_index()
```

Graph chart showing insurance non-holders by property value bracket

```
In [19]: # Graph chart
plt.figure(figsize=(8, 5))
bar_colors = ["red" if i < 2 else "lightblue" for i in range(len(avg_response))]

# Graph chart
plt.figure(figsize=(8, 5))
bars = plt.bar(
    avg_response["property_bucket"],
    (1 - avg_response["insurance_num"]) * 100,
    color=bar_colors, # updated here
)

# Assign axis labels
plt.xlabel("Pre-Storm Property Value", fontweight="bold")
plt.ylabel(
    "Percentage\nof Residents\nwithout\nInsurance\n(%)",
    fontweight="bold",
    rotation=0,
    labelpad=70,
    va="center",
    ha="left",
)

plt.title(
    "Lower-Income Residents Include Lower Proportions of Insurance Holders",
    fontweight="bold",
)

plt.ylim(0, 110)

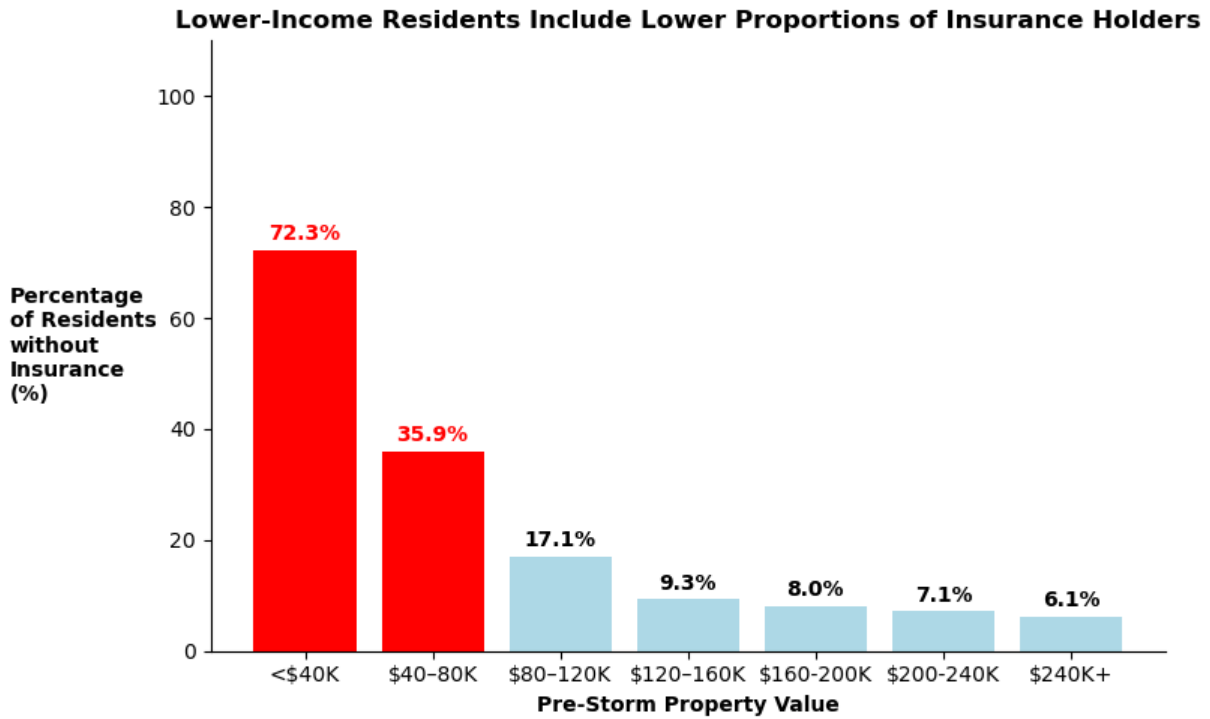
# Prepare bar labels
for i, bar in enumerate(bars):
    height = bar.get_height()
    label_color = "red" if (i == 0 or i == 1) else "black"

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 1,
        f"{height:.1f}%",
        ha="center",
        va="bottom",
        fontweight="bold",
        color=label_color,
    )

sns.despine()

plt.tight_layout()
plt.show()
```

<Figure size 800x500 with 0 Axes>



Compute Insurance Duplicative Benefits

Compute duplicative benefits and its proportion of house value

```
In [20]: roadhome["DOB Proportion"] = (
          roadhome["Closing Total DOB Amount"] / roadhome["Current PSV"]
        ) * 100
```

Take mean of insurance holders and non-insurance holders

```
In [21]: avg_dup_ben = roadhome.groupby("insurance_num")["DOB Proportion"].mean().reset_index()
```

```
In [22]: avg_dup_ben
```

```
Out[22]:
```

	insurance_num	DOB Proportion
0	0	13.535602
1	1	58.631788

Scatter Plot of Insurance and Non-insurance holders

```
In [23]: # Scatter Plot
sns.scatterplot(
    data=roadhome,
    x="Current PSV",
    y="Closing Total DOB Amount",
    hue="insurance_num",
    palette={0: "red", 1: "green"},
    alpha=0.7,
)

plt.xlabel("Property Value")
plt.ylabel("Duplicative Benefit")
plt.title("Scatter Plot by Insurance Status")
plt.legend(title="Insurance Num")
plt.show()
```

